

Graphs & Graph Algorithms

Junior ծրագրավորողի հարցազրույցի ամբողջական ուղեցույց

Այս presentation-ը ընդգրկում է graph-երի բոլոր հիմունքները junior interview-ի համար՝ ինչ տեսակի graph-եր կան, ինչպես ներկայացնել կոդում, ինչպես անցնել էլեմենտներով (BFS/DFS), ամենակարճ ճանապարհ (shortest path), և 5 դասական խնդիր՝ մանրամասն բացատրությամբ ու ամբողջական C++ կոդով:

0. Ի՞նչ է Graph-ը

Graph-ը կետերի (vertices / nodes) և դրանք կապող գծերի (edges) հավաքածու է:
Օգտագործվում է կապեր մոդելավորելու համար՝ սոցիալական ցանցեր, ճանապարհների քարտեզ, dependency-ներ և այլն:

```
G = (V, E)
V = vertices (գագաթներ)  ->  {0, 1, 2, 3}
E = edges (կողեր)         ->  {(0,1), (1,2), (2,3)}
```

Terminology, որ interview-ում հարցնում են.

Vertex / Node	-> գագաթ (կետ)
Edge	-> կող (կապ երկու գագաթի միջև)
Degree	-> գագաթին կապած edge-երի քանակ
Adjacent	-> հարևան (edge-ով միացած) գագաթներ
Path	-> գագաթների հաջորդականություն edge-երով
Cycle	-> ճանապարհ, որ վերադառնում է սկիզբ
Connected	-> ամեն գագաթ հասանելի է մյուսից

1. Graph-երի տեսակները

Undirected (չուղղորդված)

Edge-ը երկկողմանի է. եթե A-B կապ կա, կարող ես գնալ A->B և B->A:

```
A --- B
|       |
C --- D
```

Directed (ուղղորդված, digraph)

Edge-ը մեկ ուղղությամբ է. A->B չի նշանակում B->A:

```
A --> B
^       |
|       v
C <-- D
```

Weighted (կշռված)

Ամեն edge ունի կշիռ (weight/cost)՝ հեռավորություն, արժեք, ժամանակ:

```
A --5-- B
|       |
3       2
|       |
C --1-- D
```

Unweighted (առանց կշիռների)

Բոլոր edge-երը հավասար են (կշիռ = 1):

Ուրիշ կարևոր տեսակներ

Cyclic	-> ունի cycle (օղակ)
Acyclic	-> cycle չունի
DAG	-> Directed Acyclic Graph (ուղղորդված, առանց cycle)
Tree	-> connected acyclic graph, n գագաթ + $(n-1)$ edge
Dense	-> շատ edge-եր ($\sim V^2$)
Sparse	-> քիչ edge-եր ($\sim V$)

2. Graph-ի ներկայացում կողմ

Երկու հիմնական ձև կա: Interview-ում պետք է իմանալ երկուսն էլ և դրանց trade-off-ը:

Adjacency Matrix (հարևանության մատրից)

2D array `matrix[i][j] = 1`, եթե i-ից j edge կա:

```
int n = 4;
vector<vector<int>> matrix(n, vector<int>(n, 0));

void addEdge(int u, int v) {
    matrix[u][v] = 1;
    matrix[v][u] = 1;    // undirected-ի համար երկու ուղղությամբ
}
```

Space: $O(V^2)$
Edge ստուգում (u, v կա՞): $O(1)$
Հարևանների գտնել: $O(V)$
Լավ է: dense graph, արագ edge lookup

Adjacency List (հարևանության ցուցակ)

Ամեն գազաթի համար՝ իր հարևանների ցուցակ: Ամենատարածված ձևն է:

```
int n = 4;
vector<vector<int>> adj(n);    // adj[u] = u-ի հարևանները

void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);    // undirected
}
```

Weighted graph-ի համար՝ pair (հարևան, կշիռ).

```
vector<vector<pair<int,int>>> adj(n);    // adj[u] = {(հարևան, կշիռ), ...}

void addEdge(int u, int v, int w) {
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}
```

Space: $O(V + E)$

Edge ստուգում: $O(\text{degree})$

Հարևաններ գտնել: $O(\text{degree})$ (արագ)

Լավ է: sparse graph (գործնականում մեծ մասը)

Ո՞րն ընտրել

Sparse graph ($E \ll V^2$) -> Adjacency List (գրեթե միշտ)

Dense graph ($E \sim V^2$) -> Adjacency Matrix

Հաճախ edge (u,v)? հարց -> Adjacency Matrix ($O(1)$)

3. Graph Traversal — անցնել բոլոր էլեմենտներով

Երկու հիմնական ձև կա graph-ի բոլոր գագաթներով անցնելու համար՝ BFS և DFS: `visited` array-ն կարևոր է, որ cycle-ի պատճառով անվերջ չպտտվես:

BFS — Breadth First Search (լայնքով)

Անցնում է level-առ-level. սկզբում սկզբնականի բոլոր հարևանները, ապա նրանց հարևանները: Օգտագործում է **Queue** (FIFO):

```
#include <queue>

void bfs(vector<vector<int>>& adj, int start) {
    int n = adj.size();
    vector<bool> visited(n, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();
        cout << node << " ";           // մշակիր գագաթը

        for (int next : adj[node]) {
            if (!visited[next]) {
                visited[next] = true; // push-ի ՊԱՀԻՆ mark արա
                q.push(next);
            }
        }
    }
}
```

Time: $O(V + E)$

Space: $O(V)$ (queue + visited)

Օգտագործում: ամենակարճ ճանապարհի unweighted graph-ում, level-order

Քայլ առ քայլ (start=0, edges: 0-1, 0-2, 1-3).

```
Queue: [0]          visit 0 -> հարևաններ 1,2
Queue: [1,2]        visit 1 -> հարևան 3
Queue: [2,3]        visit 2
Queue: [3]          visit 3
Order: 0 1 2 3
```

DFS — Depth First Search (խորուրջամբ)

Գնում է որքան հնարավոր է խորը մեկ ճյուղով, ապա հետ գալիս (backtrack): Օգտագործում է **Stack** (կամ recursion):

Recursive (ամենապարզ)

```
void dfs(vector<vector<int>>& adj, int node, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";           // մշակիր գագաթը

    for (int next : adj[node])
        if (!visited[next])
            dfs(adj, next, visited);
}

// կանչ:
// vector<bool> visited(n, false);
// dfs(adj, 0, visited);
```


Iterative (stack-ով)

```
#include <stack>

void dfsIterative(vector<vector<int>>& adj, int start) {
    int n = adj.size();
    vector<bool> visited(n, false);
    stack<int> st;
    st.push(start);

    while (!st.empty()) {
        int node = st.top();
        st.pop();
        if (visited[node]) continue;
        visited[node] = true;
        cout << node << " ";

        for (int next : adj[node])
            if (!visited[next])
                st.push(next);
    }
}
```

Time: $O(V + E)$

Space: $O(V)$ (recursion stack / explicit stack)

Օգտագործում: cycle detection, topological sort, connected components, path finding

BFS vs DFS

BFS -> Queue, level-by-level, ամենակարճ ճանապարհ (unweighted)

DFS -> Stack/recursion, խորը գնում, cycle/topo sort/components

Երկուսն էլ: $O(V + E)$ time

4. Shortest Path — ամենակարճ ճանապարհ

Case 1: Unweighted graph -> BFS

BFS-ը երաշխավորում է ամենակարճ ճանապարհը (edge-երի քանակով), քանի որ level-առ-level է անցնում:

```
vector<int> shortestPathBFS(vector<vector<int>>& adj, int start) {
    int n = adj.size();
    vector<int> dist(n, -1);    // -1 = չհասանելի
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (int next : adj[node]) {
            if (dist[next] == -1) {    // դեռ չայցելած
                dist[next] = dist[node] + 1;    // մեկ քայլ ավելի
                q.push(next);
            }
        }
    }
    return dist;    // dist[i] = start-ից i հեռավորությունը
}
```

Time: $O(V + E)$

Աշխատում է: unweighted graph-ի համար

Case 2: Weighted graph (դրական կշիռներ) -> Dijkstra

Dijkstra-ն գտնում է ամենակարճ ճանապարհը կշռված graph-ում՝ min-heap (priority_queue) օգտագործելով: Ամեն քայլում ընտրում է ամենամոտ չմշակված գագաթը:

```

#include <queue>
#include <vector>
using namespace std;

vector<int> dijkstra(vector<vector<pair<int,int>>>& adj, int start) {
    int n = adj.size();
    vector<int> dist(n, INT_MAX);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
        greater<pair<int,int>>> pq;    // min-heap (dist, node)

    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
        auto [d, node] = pq.top(); pq.pop();
        if (d > dist[node]) continue;    // հին, բաց թող

        for (auto [next, w] : adj[node]) {
            if (dist[node] + w < dist[next]) {    // ավելի կարճ ճանապարհի գտանք
                dist[next] = dist[node] + w;
                pq.push({dist[next], next});
            }
        }
    }
    return dist;
}

```

Time: $O((V + E) \log V)$ (priority_queue-ով)

Space: $O(V)$

Աշխատում է: դրական կշիռներ (negative weight-ի դեպքում -> Bellman-Ford)

Shortest path ամփոփ

Unweighted	-> BFS	$O(V + E)$
Weighted, դրական	-> Dijkstra	$O((V+E) \log V)$
Weighted, բացասական	-> Bellman-Ford	$O(V * E)$
Բոլոր զույգերի միջև	-> Floyd-Warshall	$O(V^3)$

5. Հինգ դասական խնդիր (բացատրությամբ և կոդով)

Խնդիր 1 — Հաշվել connected component-ների քանակը

Խնդիր. Undirected graph-ում քանի առանձին «կղզի» (կապված խումբ) կա:

Գաղափար. Անցիր բոլոր գագաթներով. ամեն անգամ, երբ գտնում ես չալցելած գագաթ, դա նոր component-ի սկիզբ է. գործարկիր DFS/BFS՝ ամբողջ component-ը mark անելու, և հաշվիչը մեծացրու:

```
void dfs(vector<vector<int>>& adj, int node, vector<bool>& visited) {
    visited[node] = true;
    for (int next : adj[node])
        if (!visited[next])
            dfs(adj, next, visited);
}

int countComponents(vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            count++;           // նոր component
            dfs(adj, i, visited);
        }
    }
    return count;
}
```

Time: $O(V + E)$ | Space: $O(V)$

Խնդիր 2 — Cycle detection (undirected graph-ում)

Խնդիր. Graph-ը ունի cycle (օղակ):

Գաղափար. DFS-ի ժամանակ պահիր `parent`-ը (որտեղից եկար): Եթե հանդիպում ես արդեն visited հարևանի, որը `parent` չէ, ուրեմն cycle կա:

```

bool dfsCycle(vector<vector<int>>& adj, int node, int parent,
             vector<bool>& visited) {
    visited[node] = true;
    for (int next : adj[node]) {
        if (!visited[next]) {
            if (dfsCycle(adj, next, node, visited))
                return true;
        } else if (next != parent) {
            return true;           // visited & parent <-> cycle
        }
    }
    return false;
}

bool hasCycle(vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    for (int i = 0; i < n; i++)
        if (!visited[i])
            if (dfsCycle(adj, i, -1, visited))
                return true;
    return false;
}

```

Time: $O(V + E)$ | Space: $O(V)$

Խնդիր 3 — Grid-ում կղզիների քանակ (Number of Islands)

Խնդիր. 2D grid, **1** = ցամաք, **0** = ջուր: Քանի կղզի կա (հորիզոնական/ուղղահայաց միացած ցամաքներ):

Փառափար. Grid-ը իրականում graph է (ամեն cell = գագաթ, հարևանները՝ վերև/վար/ձախ/աջ): Ամեն չայցելած **1**-ից գործարկիր DFS՝ ամբողջ կղզին «խեղդելով» (mark 0), և հաշվիչը մեծացրու:

```

void sink(vector<vector<char>>& grid, int r, int c) {
    int rows = grid.size(), cols = grid[0].size();
    if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] == '0')
        return;
    grid[r][c] = '0';          // mark այցելած (խեղդիր)
    sink(grid, r + 1, c);
    sink(grid, r - 1, c);
    sink(grid, r, c + 1);
    sink(grid, r, c - 1);
}

int numIslands(vector<vector<char>>& grid) {
    if (grid.empty()) return 0;
    int count = 0;
    for (int r = 0; r < grid.size(); r++)
        for (int c = 0; c < grid[0].size(); c++)
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c);
            }
    return count;
}

```

Time: $O(\text{rows} * \text{cols})$ | Space: $O(\text{rows} * \text{cols})$ (recursion worst case)

Խնդիր 4 — Topological Sort (DAG-ի համար)

Խնդիր. Ուղղորդված acyclic graph-ում (dependency-ներ, օր. task-երի հերթականություն) գտիր զործարկման կարգ, որտեղ ամեն task-ը գալիս է իր dependency-ներից հետո:

Փաղափար (Kahn's / BFS). Հաշվիր ամեն գագաթի in-degree-ն (քանի edge է մտնում): Սկսիր 0 in-degree-ով գագաթներից, հեռացրու դրանք, թարմացրու հարևանների in-degree-ն:

```

vector<int> topoSort(vector<vector<int>>& adj, int n) {
    vector<int> inDegree(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : adj[u])
            inDegree[v]++;

    queue<int> q;
    for (int i = 0; i < n; i++)
        if (inDegree[i] == 0)
            q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        order.push_back(node);
        for (int next : adj[node])
            if (--inDegree[next] == 0)
                q.push(next);
    }
    // եթե order.size() < n -> graph-ը cycle ունի
    return order;
}

```

Time: $O(V + E)$ | Space: $O(V)$

Կիրառում: build systems, course scheduling, task dependencies

Խնդիր 5 — Ամենակարճ ճանապարհ maze-ում (BFS)

Խնդիր. 2D maze (0 = ազատ, 1 = պատ): Գտիր ամենակարճ քանակ քայլերի start-ից end հասնելու համար:

Գաղափար. Unweighted shortest path -> BFS grid-ի վրա: Queue-ում պահիր (row, col, distance): Առաջին անգամ, երբ հասնում ես end-ին, դա ամենակարճ ճանապարհն է:

```

int shortestPathMaze(vector<vector<int>>& maze,
                    pair<int,int> start, pair<int,int> end) {
    int rows = maze.size(), cols = maze[0].size();
    vector<vector<bool>> visited(rows, vector<bool>(cols, false));
    queue<tuple<int,int,int>> q;    // (row, col, dist)

    q.push({start.first, start.second, 0});
    visited[start.first][start.second] = true;

    int dr[] = {1, -1, 0, 0};    // վար, վեր, ձախ, աջ
    int dc[] = {0, 0, -1, 1};

    while (!q.empty()) {
        auto [r, c, d] = q.front(); q.pop();
        if (r == end.first && c == end.second)
            return d;    // հասանք -> ամենակարճն է

        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nc >= 0 && nr < rows && nc < cols
                && !visited[nr][nc] && maze[nr][nc] == 0) {
                visited[nr][nc] = true;
                q.push({nr, nc, d + 1});
            }
        }
    }
    return -1;    // չհասանելի
}

```

Time: $O(\text{rows} * \text{cols})$ | Space: $O(\text{rows} * \text{cols})$

6. Ամենահաճախ Interview հարցերը

BFS vs DFS — Լիրք որը — BFS՝ ամենակարճ ճանապարհ unweighted graph-ում, level-order; DFS՝ cycle detection, topological sort, connected components, path existence: Երկուսն էլ

$O(V+E)$:

Adjacency list vs matrix — List՝ $O(V+E)$ հիշողություն, լավ sparse graph-ի համար; Matrix՝ $O(V^2)$, edge lookup $O(1)$, լավ dense graph-ի համար:

Ինչո՞ւ է visited array-ը պետք — Cycle-ի պատճառով առանց `visited` -ի անվերջ loop կլինի, և նույն գագաթը բազմիցս կմշակվի:

BFS-ը ինչո՞ւ է տալիս shortest path (unweighted) — Level-առ-level է անցնում, ուստի գագաթին առաջին հասնելը երաշխավորված ամենաքիչ edge-երով է:

Dijkstra-ն Լիրք չի աշխատում — Բացասական կշիռների դեպքում (կարող է սխալ պատասխան տալ): Այդ դեպքում՝ Bellman-Ford:

Ինչ է DAG-ը և topological sort-ը — Directed Acyclic Graph; topo sort-ը գագաթների գծային կարգ է, որտեղ ամեն edge $u \rightarrow v$ -ի համար u -ն գալիս է v -ից առաջ: Աշխատում է միայն DAG-ի վրա:

BFS-ի queue vs DFS-ի stack — BFS՝ FIFO queue (հին էլեմենտն առաջինը); DFS՝ LIFO stack կամ recursion (վերջինն առաջինը):

Cheat Sheet

ՆԵՐԿԱՅԱՑՈՒՄ:

Adjacency List: `vector<vector<int>>` $O(V+E)$ space (sparse)
Adjacency Matrix: `vector<vector<int>>(V,V)` $O(V^2)$ space (dense)
Weighted list: `vector<vector<pair<int,int>>>`

TRAVERSAL ($O(V+E)$):

BFS -> queue, level-order, shortest path (unweighted)
DFS -> stack/recursion, cycle/topo/components

SHORTEST PATH:

Unweighted	-> BFS	$O(V+E)$
Weighted (+)	-> Dijkstra	$O((V+E) \log V)$
Weighted (-)	-> Bellman-Ford	$O(V * E)$
All pairs	-> Floyd-Warshall	$O(V^3)$

ԴԱՍԱԿԱՆ ԽՆԴԻՐՆԵՐ:

Connected components -> DFS/BFS count
Cycle detection -> DFS + parent (undirected) / recursion stack (directed)
Number of islands -> DFS/BFS on grid
Topological sort -> Kahn's (in-degree + BFS)
Maze shortest path -> BFS on grid

ՄԻՇՏ ՀԻՇԻՐ: visited array -> cycle-ից և կրկնություններից խուսափելու համար